

Interface modding guide

See also: [Interface modding](#)

İçindekiler

Basics

Understanding the files and their structure

[Templates](#)

[Types](#)

[Windows](#)

[Widgets](#)

[Updating of GUI files in game though calculation](#)

Designing your first custom window

[How to make a new window](#)

[Opening and closing your window](#)

[1. Using a variable toggle](#)

[2. Using console commands](#)

[Accessing the new window by using a button](#)

[Add a close button to your new window](#)

[Changing how our window will look in game](#)

Adding advanced UI elements

[Adding a character portrait to our new window](#)

[Adding a progress bar](#)

[Add a button for a generic action](#)

[Using buttons to nicely group things together](#)

Please help with verifying or updating older sections of this article.
At least some were last verified for [version](#) Pre-release.



Basics

The interface in EU5 is highly modable, the vanilla files can be quite tricky to understand though.

It is highly recommended to run the game in debug mode (add `-debug_mode`) to your EU5 steam starting options. With debug mode the changes you are making in the files will be visible immediately and you don't have to restart the game every time.

When opening the debug console you want to extensively use the **UI Bounds** (under 2D Tools). These option makes the gui visible and connects it the files. If you have over an UI element you will see that a black box opens and right on the top it says where this particular UI element can be found in the files. Often times you will have to cycle through some layers to get what you are interested in. Use `Alt + LeftMouseButton` to cycle through the GUI widgets. In theory you can directly open the file by pressing `Alt+ RightMouseButton`, but by default that will open the file in the Windows Notepad. You can choose which editor to use when opening files by setting the path in **Settings** → **Game** → **System** → **Editor** (double every backslash: for instance write `C:\\Program Files\\Notepad++\\notepad++.exe` instead of `C:\\Program Files\\Notepad++\\notepad++.exe`).

You also need to know that gui files both can be found in the **in_game/gui** and **main_menu/gui** folders. You might have to search both folders in order to find what you are looking for. Unfortunately as of now the UI Bounds option won't show you directly whether you need to search in **in_game/gui** or **main_menu/gui**.

Understanding the files and their structure

GUI files in EU5 are organized in a very modular way, that means that you will most of the time not find everything you need for something in the interface to change in one file only, but in many files.

The reason for this is that EU5 uses types and templates that are set up in one file and referenced in other files.

That has the advantage that you can reuse templates and types over and over again, but it also makes it more difficult to get an overview over things when starting out.

Templates

Templates are containers for properties. So **"using = template_name"** simply adds all properties (for example size) of widgets and containers, that are in the template block, to the current widget/container the template is referenced by. You can use those if you have a repeatable UI elements that always have the same structure.

```
1 template new_template_name {  
    container = { } # this is a container
```



Types

Types are widgets/containers in itself that allow reuse and overwriting properties in it. You may want to use those when you want to set up something you want to reuse later.

```

1  types my_types_general_naming {
2      type example_base_type = widget { # this is an example type of a widget
3          # Here you can define a set of overwritable properties, widgets or containers
4          block "test_block" {
5              size = { 100 100 }
6          }
7      }
8  }

```

Types can be referenced anywhere. This might look like this:

```

1  widget = {
2      example_base_type = {} # this is the type we added above
3  }

```

Windows

Windows are the top entities in the UI. Everything that can be seen in the game is inside some window. That means if you want to set up a completely new entity of some interface, you will always have to create a new window.

Widgets

Widgets are essentially smaller windows of UI elements grouped together. They are similar to windows in that, that all of the things that you want to interact with need to be in a widget.

Updating of GUI files in game though calculation

You may encounter instances where you have added scripted variables that are displayed on some window you have added and you might find that the display of those variables are not auto updated in some cases.

One of the cases where a scripted variable is not updated when a calculation changes that variable. The GUI will, in some cases, only update when the calculation of the variable is done by an event that is clicked by the player. Doing the calculation via an on action or an invisible event will give the correct results of the calculation, but the interface might not update.

This seems to only be the case for displayed variables by text as progressbars for instance update on the monthly tick.

Designing your first custom window





How to make a new window

First create a new .gui file in your **modfolder/in_game/gui** folder. You can name it whatever you want, but for this example we will call it “mymod_interface.gui”.

Then you need to add a window to this file. The name of the window is also up to you, but for this example we will call it “mymod_window”.

The most basic window will look like this in the “mymod_interface.gui” file:

```

1 window = {
2     name = "mymod_window" # this is the name we chose above
3     parentanchor = center # this determines where the window appears on your screen, in this case in the
middle
4     size = { 100% 100% } # this determines the size of your window (in this case full screen)
5     alwaystransparent = yes # this makes it so that the window does not block mouse clicks on UI below it
6 }

```

Opening and closing your window

There are two ways you can go about opening and closing your window, through a variable bound to its visibility or through console commands.

While both ways will work the same, the variable toggle is a bit more idiomatic and thus recommended.

1. Using a variable toggle

Make a new folder in your **modfolder/in_game/gui** that is called “**scripted_widgets**”. This folder is not in the vanilla files but without it your window will not appear (unless using the console command way).

There you want to create a new .txt file (make sure its encoded in UTF-8-BOM) and add your windows like this:

```
gui/mymod_interface.gui = mymod_window
```

This adds the window: “**mymod_window**”

“**mymod_interface.gui**” is the file name of the gui file the window is in.

“**gui/**” is the path from your in_game folder.

Then you will need to bind the visible property of your window to a variable like this:

```

1 window = {
2     name = "mymod_window" # This is the name we chose above
3     parentanchor = center # this determines where the window appears on your screen, in this case in the
middle
4     size = { 100% 100% } # this determines the size of your window (in this case full screen)
5     alwaystransparent = yes # this makes it so that the window does not block mouse clicks on UI below it

```



This will then open/close your window:

```
onclick = "[GetVariableSystem.Toggle('mymod_window_open'))]"
```

2. Using console commands

This will open your window:

```
onclick = "[ExecuteConsoleCommand('GUI.CreateWidget gui/mymod_interface.gui mymod_window'))]"
```

This will close your window:

```
onclick = "[ExecuteConsoleCommand('GUI.ClearWidgets mymod_window'))]"
```

If you have several scripted windows open you can also use this to close them all:

```
onclick = "[ExecuteConsoleCommand('gui.clearwidgets'))]"
```

Accessing the new window by using a button

It makes sense to create ourselves a type for our new button as this will make mod compatibility easier down the line, as most mods might want to edit the main interface files.

Lets add a type for a button that will open the idea window:

```
1 types iv2_main_idea_button {
2     type button_panel_tab_ideas =
button_panel_tab_alt { #
    "button_panel_tab_ideas" is the name of the type;
    "button_panel_tab_alt" is the type of the button
3
4     tooltipwidget = { # this is the tooltip
that is shown when hovering over the button
5         using = iv2_idea_tab_tt
6     }
7
8     onclick = "
[GetVariableSystem.Toggle('mymod_window_open'))]"
# Or via "
[ExecuteConsoleCommand('GUI.CreateWidget
gui/mymod_interface.gui mymod_window'))]" if you
want to use the console command way.
9
10    input_action = "top_left_9" # hotkey
stuff
11    use_global_input_instance = yes # hotkey
stuff
12
    using = layoutpolicy_expanding
13
    icon = { # this is the icon that is
shown on the button
```



This is how the added button looks ingame added to the menu, with the tooltip being active.



```

20         texture_density = 4
21     }
22 }
23 }

```

If we add this type like this, we only need this to add to the vanilla file (in this case the `left_panel.gui`, where the main menu interfaces is at:

```
button_panel_tab_ideas = {}
```

That is a lot easier to keep updated with changing vanilla files and to keep compatible with other mods.

Add a close button to your new window

A closing button would look like this:

```

1 widget = {
2     parentanchor = right
3     position = { 0 130 }
4     size = { 40 40 }
5
6     ui_direction_button_holder_right = {}
7
8     button_close_alt = {
9         blockoverride "close_onClick"
10        {
11            onclick = "[GetVariableSystem.Toggle('mymod_window_open')]" # Or via "[ExecuteConsoleCommand('GUI.ClearWidgets mymod_window')]" if you want to use the console command way.
12        }
13    }
14 }

```

Changing how our window will look in game

There are many ways to set up your window. The following code is merely one way to do the background art and a very basic example. The things you can do are almost limitless. Copy this part to the top of your window to see for yourself. Comment in some of the things that were commented out to see what they do and how they change the look of your window.

```

1 vbox = {
2     using = lateralview_full_margins # this is a window template
3
4     widget = {
5         using = layoutpolicy_expanding
6         alwaystransparent = no
7         allow_outside = yes
8
9         using = bg_lateralview
10
11        blockoverride "inside_color" {
12            modify_texture = {
13                #using = color_dark_purple_texture
14                #blend_mode = multiply
15            }
16        }
17
18        vbox = {
19            block "panel_header" {}

```



```

25         using = layoutpolicy_expanding
26         block "panel_content" {}
27         window_bottom_paper_template = {}
28     }
29
30     block "panel_content_side"
31     {
32         icon = {
33             layoutpolicy_vertical = expanding
34             size = { 30 -1 }
35
36             using = color_dark_brown_texture
37             #using = overlay_window
38             #using = overlay_leather
39         }
40     }
41 }
42 }
43
44 #lateralview_topbuttons = {} # this is a type
45 #window_bottom_paper_template = {} this is a type
46 }
47 }

```

Adding advanced UI elements

This section will teach you how to add some more advanced UI elements.

Adding a character portrait to our new window

You can add a character portrait to your new window and display a certain character on your interface like this:

```

1  widget = {
2      size = { 160 160 }
3      position = { 15 235 }
4
5      character_frame_small = { # the type of the frame, you can
6      search the game files for other options
7      parentanchor = left
8      blockoverride "portrait_datacontext" {
9          datacontext = "
10         [Country.MakeScope.GetVariable('Iv2_researcher_adm').GetCharacter]"
11         # this is the most important thing. This datacontext shows the
12         character in your country with the Variable "Iv2_researcher_adm"
13         You will need to set the variable by some other means, like an
14         event, a character interaction or some generic action.
15     }
16     block "character_frame_size" {
17         size = { 160 160 } # size of the frame
18     }
19     blockoverride "character_frame_texture" {
20         texture =
21         "gfx/interface/component_non_stretchable_frame_2.dds"
22     }
23     blockoverride "character_portrait_visible" {
24         visible = yes
25     }
26     blockoverride "character_portrait_ani_visible" {
27         visible = no
28     }
29 }

```



This is how an empty character portrait looks in the game.

Adding a progress bar

You can add a progress bar of a scripted variable you added, if you want to add a new resource of some sort to the game for instance. The code looks like this:

```

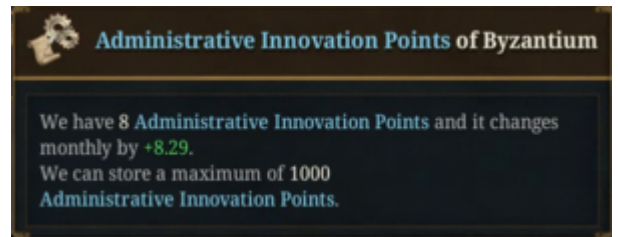
1 widget = {
2     size = { 400 20 } # size of your widget
3     position = { 40 405 } # position of this
    widget in your window
4
5     icon = { # [?] this is the frame of the
    progress bar in order for it to look nice
6     name = "frame"
7     size = { 430 20 }
8     texture =
    "gfx/interface/progressbar_frame_iv2_4.dds"
9     spriteType = Corneredstretched
10    parentanchor = center
11    position = { 7 0 }
12
13    using = button_panel_tab_icon_effect
14
15    glow = {
16        name = "drop_shadow"
17        glow_radius = 3
18        color = {0.0 0.0 0.0 1.0}
19        alpha = 0.8
20    }
21 }
22
23 hbox = {
24     layoutpolicy_horizontal = expanding
25     margin = { 2 0 }
26
27     progressbar = {
28         size = { 410 14 } # adjust this size to
        the size of your frame for it to look nice
29         using = progress_bar_blue_grey_alt # [?]
        this is a type
30         min = 0 # this is the min point of your
        bar, can also be a scripted value
31         max = "
        [FixedPointToFloat(Country.MakeScope.Var('iv2_var
        _innovation_points_limit_adm').GetValue)]" # see
        this example how to set the max point of your
        progress bar via a scripted value
32         value = "
        [FixedPointToFloat(Country.MakeScope.Var('iv2_var
        _innovation_points_adm').GetValue)]" # this value
        determines what is shown on the progress bar.
33         noprogresstexture =
        "gfx/interface/progressbars/progress_gray_bg.dds"
34         block "progress_setup" {}
35
36         tooltipwidget = { # this is a tooltip
        shown when you hover over the bar
37             using =
        iv_scrollbar_adm_tooltip_template
38         }
39     }
40 }
41 }
```



This is how a character portrait in the game looks like with an added character. The character you see here has the variable that was added by a different action



This is how the example progressbar looks ingame.



This is how the tooltip of this progressbar looks.

! a button for a generic action

Adding a button for new custom added generic actions looks like this:

This is how this generic action button looks ingame.

```

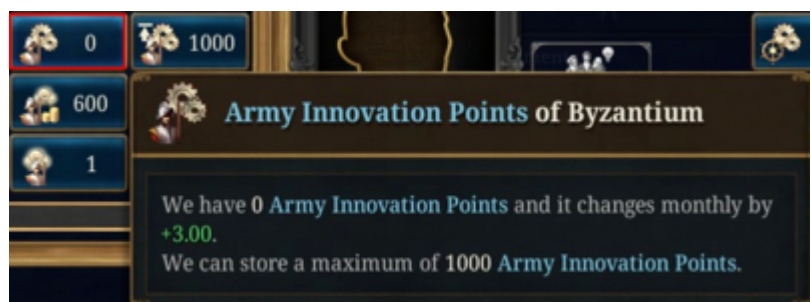
1  card_header_action_button_04 = {
2      size = { 80 40 }
3      position = { 180 300 }
4
5      actor = "[Player]" # this references who is clicking the button; Player in
        this case is the country
6      parameter = {
7          parameter_name = "target" #this references who gets the effect when the
        button is pressed
8          parameter_value = "[Country]"
9      }
10
11     left_click_and_hold_action = { action_name = "iv2_ga_hire_researchers_adm" } #
        this references your generic action
12
13     tooltipwidget = { # this is a tooltip again
14         using = iv2_tooltip_template_res_adm
15         blockoverride "tooltip_content" {
16             using = action_tooltip_inner_content
17         }
18     }
19
20     icon = {
21         size = { 36 36 }
22         parentanchor = center
23         texture = "gfx/interface/icon_hire_adm.dds"
24         texture_density = 2
25         using = button_panel_tab_icon_effect
26         glow = {
27             name = "drop_shadow"
28             glow_radius = 3
29             color = {0.0 0.0 0.0 1.0}
30             alpha = 0.8
31         }
32     }
33 }

```

Using buttons to nicely group things together

Buttons can also be used to efficiently display information that is ultimately not clickable. Like this you have nice area to hover over to display a tooltip.

This button for instance displays an icon and a text (in this case a dynamic variable).



The first hbox you see is the icon, the second hbox is the text. They are all inside a button resulting in that all of the red area is hoverable and the tooltip will appear.

```

1  button_default = {
2      size = { 80 40 }
3      position = { 1300 280 }
4
5      tooltipwidget = {
6          using =
        iv_scrollbar_army_tooltip_templa
        te
7      }
8
9      hbox = {
10         position = { -20 1 }
11         margin = { 20 0 }

```



```

16         texture_density = 4
17
18         using = button_panel_t
19
20         glow = {
21             name = "drop_shado
22             glow_radius = 3
23             color = {0.0 0.0 0
24             alpha = 0.8
25         }
26     }
27 }
28 hbox = {
29     position = { 15 0 }
30     text_single = {
31         name = "iv2_text"
32         text = "IV_IDEAGROUP_C
33     }
34 }
35 }

```

Setting up Tooltip Widgets

Setting up tooltip widgets can be quite tricky, but basically every information you want can be displayed with the right hoverable tooltip.

Referencing tooltips in the files

Tooltips can be placed virtually anywhere. They can be on an icon, on a button, on some text, on a scrollbar. You might have seen some examples already in the examples provided above.

Lets take a look at their basic structure in the files:

First example:

```

1 tooltipwidget = {
2     using = iv_scrollbar_navy_tooltip_template
3 }

```

Second example:

```

1 tooltipwidget = {
2     using = iv2_tooltip_template_res_navy # this is a tooltip template
3     blockoverride "tooltip_content" { # this changes the basic tooltip template in this special case
4         using = action_tooltip_inner_content
5     }
6 }

```

Templates are excessively used with tooltip widgets in vanilla EU5, which is why it can be quite tricky to track them down to the source. The second example shows that we can change our basic template within a tooltip widget without creating a new template.

This is how the templates of the examples above look:



```
5      using = tooltip_title_icon_size
6      texture =
7      "gfx/interface/icon_hire_navy.dds"
8
9      glow = {
10          name = "drop_shadow"
11          glow_radius = 3
12          color = {0.0 0.0 0.0 1.0}
13          alpha = 0.3
14      }
15  }
16
17  blockoverride "title_text" {
18      text = "iv2_ga_hire_researchers_navy"
19  }
20
21  blockoverride "concept_link" {
22      text = "[iv2_researcher/e]"
23  }
24  }
25 }
```



This is how the tooltip looks ingame. Note: The additional information you see (Effects, Conditions) are all coming from the general action that this tooltip is referencing)

With this template you can see that the vanilla “ContextualTooltipType” was used and changed for our special tooltip to work to our liking. Sometimes it matters which vanilla tooltip type you choose as not every information can be referenced everywhere. Depending on what you choose you might encounter things not working as you intended.

References

Modding		Return to top
Documentation	Defines • Effects • Scopes • Scope links • Triggers Colors • Macros • Mean time to happen • Modifier types • On actions • Script value • Variables GUI script • Localization	
Scripted content	Actions • Disasters • Events • Missions • Modifiers • Scripted gui • Setup • Situations • Customizable localization	
Scripted types	Advances • Art • Buildings • Bureaucracies • Casus belli • Characters • Concepts • Countries • Culture • Diplomacy • Diseases • Estates • Goods • Institutions • International organizations • Laws • Movements • Peace treaties • Pops • Religion • Subject types • Traits • Units • Wargoals	
Map	Map • Map modes • Terrain	
Graphics	3D Models • Interface • Graphical assets • Fonts • Flags	
Audio	Music • Sound	
Other	AI • Console commands • Checksum • Mods • Mod compatibility • Mod structure • Troubleshooting	
Guides	Interface modding guide • Mod translation • Save-game editing • Settlement position modding guide	
Tools	Arcanum • PDX DeepL • PDX Workshop Manager • Community Mod Toolkit • Add Your Tool to the Wiki	

"https://eu5.paradoxwikis.com/index.php?title=Interface_modding_guide&oldid=32980" sayfasından alınmıştır

Sayfa en son 18.51, 7 Mart 2026 tarihinde değiştirildi.

Aksi belirtilmedikçe içeriğin kullanımı Attribution-ShareAlike 3.0 lisansı kapsamında uygundur.





GAMES

- Discover
- Our Brands
- Browse
- Play on Paradox technology

COMMUNITY

- Paradox Forums
- Paradox Wikis
- Support

ABOUT

- News
- About us
- Careers
- Join our playtests
- Media contact

SOCIAL MEDIA

- Terms & Policies
- Legal Information
- EU Online Dispute Resolution
- Report Illegal Content
- Frequently Asked Questions
- Paradox Interactive corporate website

